LAB 6

COMMUNICATION USING THE UART, PART II, AND INTERRUPTS

INTRODUCTION

This week in lab you will get familiar with a key concept in embedded systems – interrupts. You will continue to use the UART, and instead of using polling as the method of input/output, you will use interrupts. You will remove the blocking function for receiving data, and instead trigger an interrupt when a byte is received in the UART. Doing so, your code will not have to busy-wait in a loop waiting for a key to be pressed in PuTTY, possibly ignoring other work to do, and instead your code will be notified when the character from the keystroke has been received. This functionality will be very useful for the lab project.

REFERENCE FILES

The following reference files will be used in this lab:

- lab6_template.c, contains a main function template to be used as needed with uart.h and uart.c
- uart.h, header file for uart.c
- uart.c, program file with partially implemented functions (use your uart.c file from Lab 5)
- lab6-interrupt_template.c, contains a main function template to use with uart-interrupt.h and uart-interrupt.c files
- uart-interrupt.h, header file for uart-interrupt.c
- uart-interrupt.c, program file with partially implemented functions (**you will implement the functions in this lab**)
- cyBot_Scan.h, header file for pre-compiled library for CyBot sensor scanning
- lcd.c, program file containing various LCD functions
- lcd.h, header file for lcd.c
- timer.c, program file containing various wait commands
- timer.h, header file for timer.c
- TI Tiva TM4C123G Microcontroller Datasheet
- TI TM4C123G Register Definitions C header file: REF_tm4c123gh6pm.h
- GPIO and UART register lists and tables: GPIO-UART-registers-tables.pdf
- Reading guides for GPIO, UART, and interrupts, as needed, e.g., reading-guide-interrupts.pdf

The code files are available to download.

PRELAB

See the prelab assignment in Canvas and submit it prior to the start of lab.

STRUCTURED PAIRING

You are expected to continue to use structured pairing in this lab and in future labs. It was introduced in Lab 2.

PART 1: UART NON-BLOCKING RECEIVE

Suppose you press 'g' in PuTTY to send a go command to the CyBOT to initiate a sensor scan. Now, suppose you want to stop the scan, such as when you press 's' in PuTTY.

Think about whether this is possible with a blocking receive function. Note that the `cyBot_getByte()` and `uart_receive()` functions used thus far are programmed as a blocking receive. What is the effect of using a blocking function on the rest of your program? What happens when your program busy-waits for a character to be received from PuTTY?

Try writing a non-blocking receive function that returns the character received, or returns 0 if no character was received. Make a copy of the `uart_receive()` function and rename it as `uart_receive_nonblocking()`. Modify this new function so that it is nonblocking (i.e., does not busy-wait on the RX flag). Note that the function will still need to check the flag. Use this new function to help you implement a stop command.

CHECKPOINT:

Send an 's' command to the CyBot from PuTTY to stop a scan, otherwise the scan should operate as usual.

PART 2: UART RECEIVE USING INTERRUPTS

As shown in part 1, if the `uart_receive()` function is implemented as a blocking receive, that means the function is waiting in a loop until the UART receives a byte. This prevents a program from doing other tasks while waiting. A nonblocking receive solves that problem. However, even with a nonblocking receive, there are other issues to consider. For example, the behavior of your program depends on how frequently it checks for the stop command, other code your program is executing, and how quickly it reacts to stop the scan. Whether due to slow input devices or slow software design, your program could be unresponsive to other tasks. One way to make programs more efficient and responsive is to use interrupts.

Suppose that instead of having your program wait for a byte to be received, the bytes are automatically received and echoed, independent of your program. If needed by your program, the bytes could also be saved in a buffer, and your program would simply read bytes from the buffer when needed while doing other tasks.

This approach is shown below in Figure 12.2 from the VYES book (here "Fifo" means a character buffer).

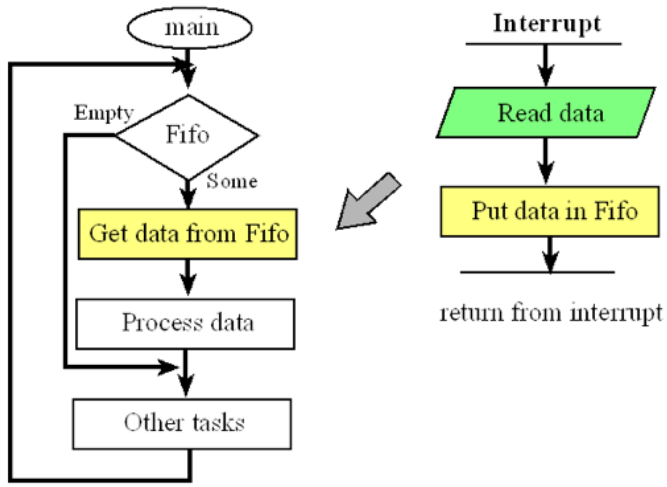

Figure 12.2. For an input device we can use a FIFO to pass data from the ISR to the main program.

In this part of the lab, you will re-write your code to use interrupts. Simply put, **your program will no longer call any version of the uart_receive() function. A character will be received by another function called an interrupt handler or interrupt service routine (ISR)**. While you as a programmer write the ISR, you do not call it. It is executed automatically when a receive/RX interrupt occurs and is processed by the interrupt system. **To repeat, <u>your program does not call</u> the interrupt handler (or ISR).**

Using interrupts requires the following steps:

1. Enable interrupts from UART1 RX (receiving a byte triggers a UART interrupt)
2. Enable the interrupt controller (NVIC, which stands for Nested Vector Interrupt Controller) to process UART1 interrupts and tell the CPU which interrupt handler function (or ISR) to execute
3. Enable interrupts in the system (letting the CPU suspend executing your main program, and go execute the ISR)
4. Declare volatile global variables as needed for the ISR to share data with your main program (an ISR does not return a value, and does not have arguments, so global variables are used)
5. Write an interrupt handler function (ISR) to "service the interrupt"
6. Write the main program so that it uses global variables as needed to "talk" with the ISR

Most of these steps have been implemented for you in this part of the lab. For now, our goal is to introduce you to interrupts so you become aware of what they are, why they are important, and generally how they work. You should gain some awareness of interrupt processing features of the TM4C123 microcontroller, even if you don't understand all of the coding details. Your lab project will involve the use of several interrupts.

****Note**: In order to use interrupts, you must include driverlib/interrupt.h and stdbool.h. You should use the following header files.

```
#include <stdbool.h>
#include "driverlib/interrupt.h"
```

Tip: In order to view functions that may be useful for configuring interrupts, once the interrupt library is included as a header file, you can use Ctrl + Click on the name of the include file to open it. This is also useful for viewing the header file containing the register macros.

Refer to the following "interrupt" code files for Lab 6 and browse through all of them:

- lab6-interrupt_template.c
- uart-interrupt.h
- uart-interrupt.c

First, update the code in uart-interrupt.c – start by copying code written (and tested) in Lab 5.

1. Copy code from uart.c to the uart-interrupt.c file as appropriate based on the initialization code previously written for the UART (i.e., from uart.c, where you replaced the "???" placeholders).

   Also copy code to implement the functions (i.e., the "TODO" placeholders). To repeat, you are copying uart.c code already written from Lab 5.

2. Notice the **new "?????" placeholders** in the uart-interrupt.c file (i.e., 5 question marks). **You need to write code for these.** The comments in the code are intended to provide guidance.

3. Start with the code in the uart_interrupt_init() function to initialize UART1 to use receive (RX) interrupts. Note that **you will only be using RX interrupts**. You will not use interrupts for transmit (TX). **See steps 1-3 above in purple**.
   a. You will need to look up specific registers in the Tiva datasheet to complete the code.
   b. The NVIC enable registers (NVIC_ENx_R) appear complicated in the datasheet. Instead, all you need to know is that there is <u>one enable bit in an NVIC EN register for each device that can interrupt</u>. Devices that can interrupt are numbered starting with 0 (called the Interrupt Number, or IRQ Number, where IRQ stands for Interrupt ReQuest). The Interrupt/IRQ Number for UART1 is 6. This is shown in Table 2-9 in the datasheet. That means that <u>bit 6 in the NVIC enable register is used to enable or disable UART1 interrupts</u>.

4. Next see the UART1_Handler() function, which is the interrupt handler or interrupt service routine (ISR). You, as an embedded programmer, write the code for this function based on how you want to respond to the UART1 interrupt. However, your program does not call this function. The interrupt system of the microcontroller tells the CPU if and when to execute this function. **Read those last three sentences again; this is a key principle of interrupt-driven I/O.** ☺

   >>> You write the code for the interrupt handler (ISR) function.
   >>> Your program does not call this function.
   >>> The interrupt system of the microcontroller tells the CPU to execute this function.

   Some ISR code has been written for you. **See step 5 above in purple**.
   a. Start by writing code for the "?????" placeholders in the ISR. **Simply replacing these placeholders should make the ISR fully functional to receive and echo characters from PuTTY. You don't need to modify/add any other code in the ISR for now.**
   b. Test that receiving and echoing work using interrupts before adding any new functionality that uses the global shared variables.

c. In other words, at this point, once you have replaced all placeholders (???, ?????, TODO), you can build your program with the main function in lab6-interrupt_template.c, and it should support communication between PuTTY and the CyBot.

Remember, previously written UART code to receive bytes from PuTTY used a polling method for reading data. In this part of the lab, you are using an interrupt method for reading data.

NOTE: Do not print to the LCD screen from inside an ISR. Printing is a slow operation. An ISR must execute quickly.

Instead, you could print characters to the LCD screen from your main program. If so, then the main program needs to know what character was received. Remember that an ISR does not have parameters or return values. So, how does your main program know what character was received? A global variable would be used. For example, declare a single global variable for the character received. The ISR writes the character received to the global variable, and the main program reads the variable and prints the character to the LCD screen. You could repeatedly print the variable without worrying about whether it's a new key pressed. Or you could assign the null byte to the global variable after printing it. When a key is pressed and the new character is printed, this indicates that the ISR was executed. You can also watch the global variable in the debugger. In short, it's a useful exercise to add a global variable in this part of the lab, before trying Part 3. A key question: where and how is a global variable declared when used by an ISR and main program?

CHECKPOINT:

Verify that the receive/echo functionality works using interrupts. The CyBot should receive characters from PuTTY and send each character it receives back to PuTTY to be displayed. Your program must not call any version of the uart_receive() function. Using the debugger, verify that the interrupt handler is being executed.

(OPTIONAL) PART 3: USING INTERRUPTS TO RECEIVE SENSOR SCAN COMMANDS (2 BONUS POINTS)

Recall the go ('g') and stop ('s') commands from part 1. In this part of the lab, implement one or both of these commands using interrupts.

For example, you could implement the go command using the original blocking receive function, i.e., call uart_receive() in your program until 'g' is typed and then start a scan. However, instead of checking for the stop command by calling a function, as you did in part 1, let the stop command be processed as an interrupt. The comments in the code provide guidance on how to do this.

Notice in uart-interrupt.c, there are global variables. You can use these, or declare your own.

```
// value for special character to be used as a command
// e.g., initialize here or assign in main program, such as 's'
volatile char command_byte = -1;
// flag to tell the main program a special command was received
volatile int command_flag = 0; //0 means command not received
```

Next look at the interrupt handler code.

```
    {
            //AS NEEDED
            //code to handle any other special characters
            //code to update global shared variables
            //DO NOT PUT TIME-CONSUMING CODE IN AN ISR

            if (byte_received == command_byte)
            {
              command_flag = 1;
            }
        }
```

If you are only processing one command using interrupts, e.g., stop command using character 's', the above code should suffice. If you want more commands to be interrupt-driven, then you will need additional comparisons with all possible commands.

Keep in mind that the global variables used here in the interrupt handler and in the main program provide a way for these code elements to "talk" to each other about a command that has been received.

Note the comment: do not put time-consuming code in an ISR. Interrupt handlers are supposed to get their work done as quickly as possible so that a) the CPU can resume executing your program, and b) the next interrupt is processed promptly. **For example, do not call the scan function in the ISR (call it in your main program). Do not busy-wait on any other input in the ISR.**

Finally, take a look at lab6-interrupt_template.c. You did not need to modify the main program for part 2, and now you will add code. Read through the "YOUR CODE HERE" comments and add new functionality.

Here is some functionality you may want in the main while(1) loop:

- Code from part 1 to start the scan when the go command is received
- Scanning and displaying sensor data in PuTTY
- Checking the command_flag to see if the stop command was received, and if so, stop the scan
- Reset the command_flag to 0

There are many ways to program this, and the code and comments suggest one approach.

CHECKPOINT:

Start and stop a scan using commands from PuTTY, with at least one command being interrupt-driven.

DEMONSTRATIONS:

1. **Functional demo of a lab milestone –** Specific milestone to demonstrate is Part 2.
2. **Debug demo using debugging tools to explain something about the internal workings of your system –** The TA will announce any specific debugging requirements at the start of lab; otherwise you will create your own debug demo based on your needs and interests in the lab.

3. **Q&A demo showing the ability to formulate and respond to questions –** This can be done in concert with the other demos.